

Linear Regression

Nishanka Das

Department of Computer Science
Ramakrishna Mission Vivekananda Centenary College, Rahara

July 30, 2026

For any mistakes found please feel free to mail at:
`nishanka.das.rkmvcc@gmail.com`

Abstract

This document presents a mathematically rigorous exploration of regression theory, tracing the progression from canonical linear models to non-linear feature mappings via radial basis functions. We begin by formulating the linear regression framework, deriving the Mean-Squared-Errors (MSE) loss function from both probabilistic (Maximum Likelihood Estimation) and deterministic perspectives.

We explicitly evaluate two primary optimization paradigms: the exact closed-form analytical solution via the Normal Equations and the iterative first-order optimization via batch Gradient Descent. The computational trade-offs, scalability limits, and convergence properties of both approaches are thoroughly analyzed. Finally, to bridge theoretical formulations with practical software execution, we provide step-by-step algorithmic structures and procedural pseudocode for implementing both analytical and iterative solvers from scratch.

1 Linear Regression

The goal of regression is to predict the value of one or more continuous target variables given the value of a D -dimensional vector of input variables. We have N observations $\{x_n\}$ together with corresponding target values t . The target is to predict value t for a new value x . To do this, we formulate a function $y(x, w)$ whose values for new inputs x constitute the predictions for the corresponding values of t , where w represents a vector of parameters.

$$y(x, w) = w_0 + w_1x_1 + \cdots + w_Dx_D \quad (1)$$

is the simplest form of regression. Here, $x = (x_1, x_2, \dots, x_D)^T$. The model is a linear function of the parameters $w_0, w_1, \dots, w_D$. This specific model is called linear regression.

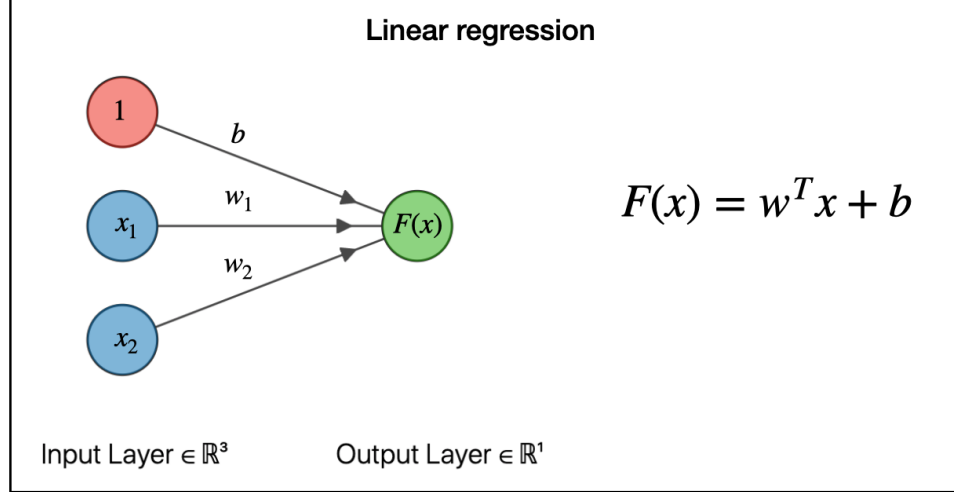
Basis Functions

The class of models can be extended from Eqn. (1) by considering linear combinations of fixed non-linear functions of input variables:

$$y(x, w) = w_0 + \sum_{j=1}^{M-1} w_j\phi_j(x) \quad (2)$$

where $w = (w_0, w_1, \dots, w_{M-1})^T$ and $\Phi = (\phi_0, \phi_1, \dots, \phi_{M-1})^T$. Here the total number of parameters is M . The parameter w_0 is called a bias parameter, and $\phi_0(x) = 1$. The model can be rewritten as:

$$y(x, w) = \sum_{j=0}^{M-1} w_j \phi_j(x) = w^T \phi(x) \quad (3)$$



$$F(x) = w_1 \cdot x_1 + w_2 \cdot x_2 + 1 \cdot b$$

Figure 1: Linear combination of basis functions $\phi_j(x)$ weighted by parameters w_j .

Different Basis Functions

- **Polynomial Basis Function:**

$$\phi_j(x) = x^j$$

- **Gaussian Basis Function:**

$$\phi_j(x) = \exp \left\{ -\frac{1}{2} \left(\frac{x - \mu_j}{s} \right)^2 \right\}$$

where μ_j is the location parameter or center of the j -th basis function in the input space. It controls where along the x -axis the specific basis function is activated. The parameter s is the scale parameter (often related to the variance σ^2 or standard deviation σ of Gaussian). It governs the width or spread of the basis function.

- **Sigmoid Basis Function:**

$$\phi_j(x) = \sigma \left(\frac{x - \mu_j}{s} \right)$$

where $\sigma(a)$ is the logistic sigmoid function:

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

The function $\tanh(a) = 2\sigma(2a) - 1$ is similar to the logistic sigmoid.

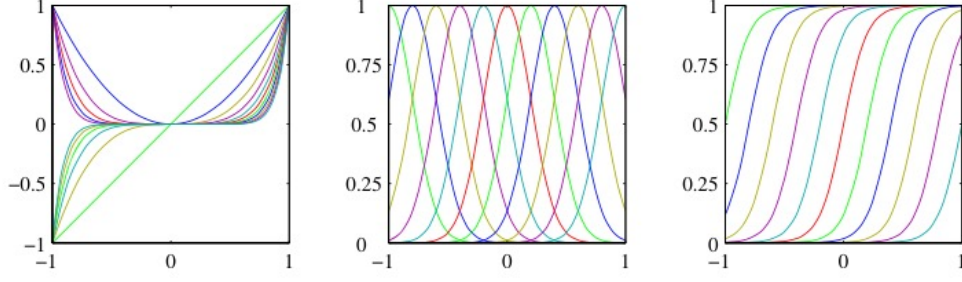


Figure 2: Here is the image of Polynomial basis function, Gaussian Basis Function and Sigmoid Basis Function

2 Mathematical Formulation of Linear Regression

Linear regression learns a mapping $f : \mathcal{X} \rightarrow \mathcal{Y}$ from input features to output target variables. For n training samples with d features, let:

- $X \in \mathbb{R}^{n \times d}$ be the design matrix (input features)
- $Y \in \mathbb{R}^n$ be the target vector
- $w \in \mathbb{R}^{d \times 1}$ be the weight vector
- $b \in \mathbb{R}$ be the bias term

The model hypothesis is:

$$f(x) = w^T x + b$$

Loss Function

The objective is to find w that minimizes the error function $E(w)$:

$$E(w) = \|Xw - Y\|^2 = (Xw - Y)^T (Xw - Y)$$

Expanding the matrix multiplication:

$$E(w) = w^T X^T X w - w^T X^T Y - Y^T X w + Y^T Y$$

Now let's look at the middle terms:

$$w^T X^T Y \implies (1 \times d) \times (d \times n) \times (n \times 1) = (1 \times 1) \quad (\text{a scalar})$$

$$Y^T X w \implies (1 \times n) \times (n \times d) \times (d \times 1) = (1 \times 1) \quad (\text{a scalar})$$

Using the scalar transpose identity (any scalar is equal to its own transpose, $\alpha = \alpha^T$):

$$(w^T X^T Y)^T = Y^T (w^T X^T)^T = Y^T (X^T)^T (w^T)^T = Y^T X w$$

So, $w^T X^T Y = Y^T X w$. Therefore:

$$E(w) = w^T X^T X w - 2w^T X^T Y + Y^T Y$$

To get the minimum error, we take the derivative with respect to w and set it to zero.

From matrix calculus, $\nabla_w (w^T A w) = 2Aw$ for a symmetric matrix A . Here, let $A = X^T X$. Notice that A is symmetric because:

$$A^T = (X^T X)^T = X^T X = A$$

Taking the derivative gives:

$$\frac{\partial E(w)}{\partial w} = 2X^T X w - 2X^T Y$$

Now we have two methods to solve for w :

Case 1: Closed-Form Solution

Set the gradient $\nabla_w E(w) = 0$:

$$\begin{aligned} 2X^T X w - 2X^T Y &= 0 \\ X^T X w &= X^T Y \end{aligned}$$

Assuming $X^T X$ is invertible:

$$w^* = (X^T X)^{-1} X^T Y$$

Note: This method calculates the exact global minimum in a single operation. No learning rate or training epochs are needed.

Case 2: Iterative Gradient Descent

When $X^T X$ is too large to invert efficiently (or n is very large), update weights iteratively over epochs using a learning rate η :

1. **Initialize:** Set $w^{(0)}$ (small random values).
2. **Compute Gradient:** At step t , evaluate the gradient:

$$\nabla_w E(w^{(t)}) = 2(X^T X w^{(t)} - X^T Y)$$

3. **Update Weight:**

$$w^{(t+1)} = w^{(t)} - \eta \nabla_w E(w^{(t)})$$

4. **Repeat** for a fixed number of epochs or until it converges ($\|\nabla_w E(w)\| < \epsilon$).

Finally, for a new input x , calculate the predicted value $\hat{y}$:

$$\hat{y} = w^T x + b$$

3 Algorithmic Implementations

To bridge theoretical derivations with software execution, the procedural logic for both optimization techniques is detailed below.

Algorithm 1: Closed-Form Solution (Normal Equations)

Algorithm 1 Closed-Form Linear Regression Solver

Require: Design Matrix $X \in \mathbb{R}^{n \times d}$, Target Vector $Y \in \mathbb{R}^{n \times 1}$

Ensure: Optimal Weight Vector $w^* \in \mathbb{R}^{d \times 1}$

```
1: procedure NORMALEQUATIONS( $X, Y$ )
2:    $A \leftarrow X^T X$  ▷ Compute  $d \times d$  Gram matrix
3:    $b \leftarrow X^T Y$  ▷ Compute  $d \times 1$  target projection
4:   if  $\det(A) = 0$  then
5:      $A^{-1} \leftarrow$  Pseudo Invert Matrix  $X^T X$ 
6:   end if
7:   if  $\det(A) \neq 0$  then
8:      $A^{-1} \leftarrow$  Invert( $A$ ) ▷ Invert  $d \times d$  matrix via LU/Cholesky
9:   end if
10:   $w^* \leftarrow A^{-1} \cdot b$  ▷ Compute parameters  $w^* = (X^T X)^{-1} X^T Y$ 
11:  return  $w^*$ 
12: end procedure
```

Algorithm 2: Iterative Batch Gradient Descent

Algorithm 2 Batch Gradient Descent Linear Regression

Require: $X \in \mathbb{R}^{n \times d}$, $Y \in \mathbb{R}^{n \times 1}$, Learning rate $\eta > 0$, Tolerance $\epsilon > 0$, Max Epochs K

Ensure: Optimized Weight Vector $w \in \mathbb{R}^{d \times 1}$

```
1: procedure GRADIENTDESCENT( $X, Y, \eta, \epsilon, K$ )
2:   Initialize  $w^{(0)} \sim \mathcal{N}(0, 0.01 \cdot I_{d \times 1})$  ▷ Small random initial weights
3:    $t \leftarrow 0$ 
4:   repeat
5:      $\hat{Y} \leftarrow Xw^{(t)}$  ▷ Compute predictions for all  $n$  samples
6:      $E \leftarrow \hat{Y} - Y$  ▷ Compute error residual vector
7:      $g^{(t)} \leftarrow 2 \cdot X^T E$  ▷ Compute gradient  $\nabla_w E(w) = 2(X^T X w - X^T Y)$ 
8:      $w^{(t+1)} \leftarrow w^{(t)} - \eta \cdot g^{(t)}$  ▷ Update weight parameters
9:      $t \leftarrow t + 1$ 
10:  until  $\|g^{(t)}\|_2 < \epsilon$  or  $t \geq K$  ▷ Convergence check
11:  return  $w^{(t)}$ 
12: end procedure
```
